



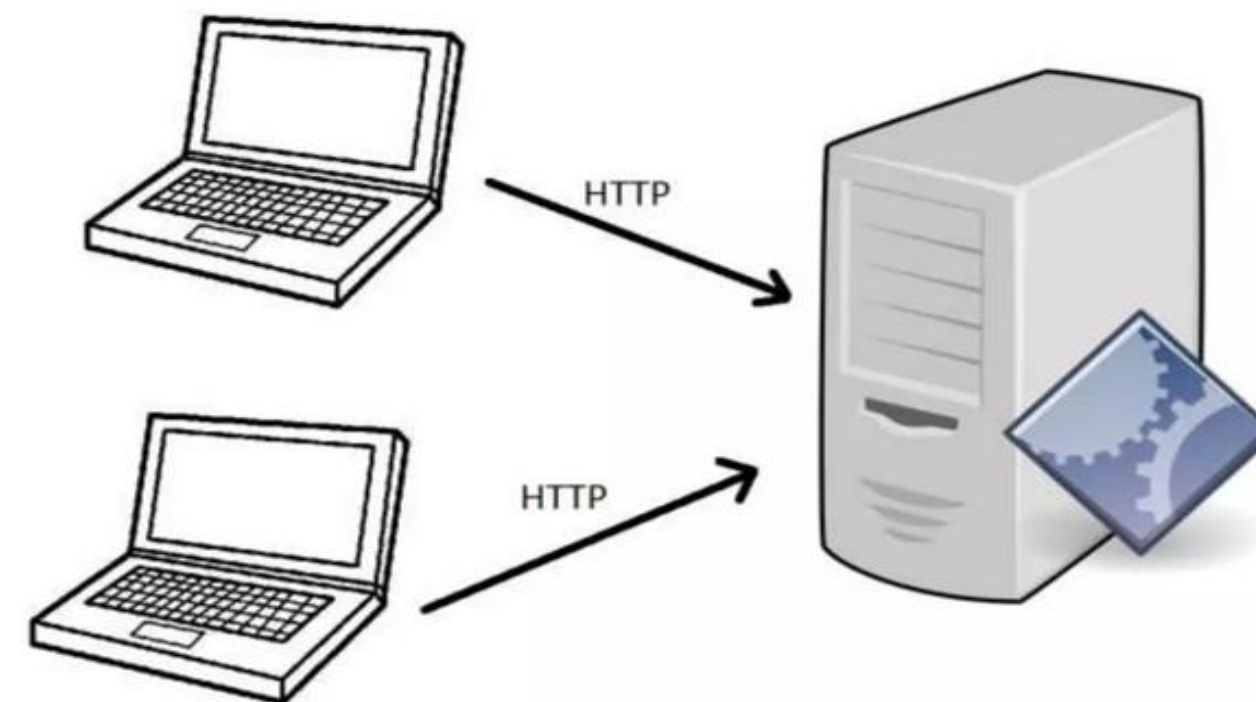
Web API

Autor: Sediangani Sofrimento
Área: DET

- ❑ **Protocolo HTTP**
- ❑ **URI vs URN vs URL**
- ❑ **Recursos**
- ❑ **Tipos de Serviço**
- ❑ **StateFul vs Stateless**
- ❑ **REST**
- ❑ **SOAP vs REST**
- ❑ **Verbos HTTP**



- ❑ Conjunto de regras de transmissão de dados que permitem que máquinas com diferentes configurações possam se comunicar com um mesmo “idioma”
- ❑ Protocolo baseado em requisição e resposta entre Cliente e Servidor
- ❑ Quando o Client solicita um recurso na internet ele envia um **pacote de dados** com cabeçalhos (Headers) a um **URI** (ou **URL**) e o destinatário (provavelmente um Servidor) vai devolver uma resposta que pode ser um **recurso** ou outro cabeçalho.



➤ URL – Uniform Resource Locator

- Refere-se ao **local** ou **host** onde se encontra o recurso
- Exemplo
 - isptec.co.ao
 - abola.pt
 - invoiceexpress.com
 - invoicingplan.com
- Ao acessarmos a esses endereços caímos no servidor onde se encontram os recursos.
- A **URL** é parte da **URI**



➤ URN – Uniform Resource Name

- Refere-se ao **nome do recurso** que pretendemos aceder e que também fará parte da **URI**
- Exemplo
 - home.html
 - contacto.php
 - Servicos.html
- É comum associarmos o URN a página que estamos a ceder
 - Ex: /api/v01/exemplo/utilizadores



❖ URI – Uniform Resource Identifier

☐ Refere-se ao Identificador do Recurso

- Pode ser: uma imagem, uma página etc...

☐ Exemplos de **URI**

- [https://pt.wikipedia.org/wiki/Hypertext Transfer Protocol](https://pt.wikipedia.org/wiki/Hypertext_Transfer_Protocol)
- <https://www.invoicingplan.com/servicos/>

☐ A **URI** é composta pelo **protocolo**(https://) + a **URL** + **URN**

<https://www.dominio.com.ao/webservicerest/?getClients=true&id=2>



❑ Devem ter nomes e endereços definidos podem ser físicos ou não:



facebook



- ❑ Servidores Web entendem URI
- ❑ URI possui uma estrutura definida
- ❑ Métodos possuem uma assinatura
- ❑ Podemos mapear URI para métodos



- Padrão arquitetural
- Criado por Roy Fielding em 2000
- Baseado no conceito de **URIs**
- Escalável
- Verbos HTTP
- StateLess
 - Não é mantido o estado das requisições



XML

```
2 <?xml version="1.0" encoding="UTF-8"?>
3 <ContactsResponse xmlns:i="http://www.w3.org/2016">
4   <Contactos>
5     <Contacto>
6       <PrimeiroNome>Oscar</PrimeiroNome>
7       <UltimoNome>Cardozo</UltimoNome>
8       <Idade>36</Idade>
9     </Contacto>
10    <Contacto>
11      <PrimeiroNome>Pablo</PrimeiroNome>
12      <UltimoNome>Aimar</UltimoNome>
13      <Idade>33</Idade>
14    </Contacto>
15    <Contacto>
16      <PrimeiroNome>Lionel</PrimeiroNome>
17      <UltimoNome>Messi</UltimoNome>
18      <Idade>29</Idade>
19    </Contacto>
20  </Contactos>
21 </ContactsResponse>
```

JSON

```
3 {
4   "contactos": [
5     {
6       "PrimeiroNome": "Oscar",
7       "UltimoNome": "Cardozo",
8       "Idade": 34
9     },
10    {
11      "PrimeiroNome": "Pablo",
12      "UltimoNome": "Aimar",
13      "Idade": 33
14    },
15    {
16      "PrimeiroNome": "Lionel",
17      "UltimoNome": "Messi",
18      "Idade": 29
19    }
20  ]
21 }
```


□ Indicam qual acção será executada

- **GET**
- **POST**
- **PUT**
- **DELETE**



□ **GET** – Responsável por ir buscar informações através de uma **URI**

- <http://www.server.com/produtos>
- <http://www.server.com/produtos/105>
- <http://www.server.co.ao/produtos/notebook>

□ **POST** – Responsável por enviar informações através de uma **URI**, com o conteúdo embutido no body(corpo) da requisição

- <http://www.server.co.ao/produto>

```
{ "Nome": "Mouse", "Preco" : 50.0 }
```



❑ **DELETE** – Responsável por remover informações através de uma **URI**

- **`http://www.server.com/produtos/7`**

❑ **PUT** – Responsável por actualizar informações através de uma **URI**, com o conteúdo embutido no body(corpo) da requisição

- **`http://www.server.co.ao/produto/105`**

{ “Nome”: “Mouse”, “Preco” : 50.0 }



- ❑ O que é uma Web API
- ❑ Configurações



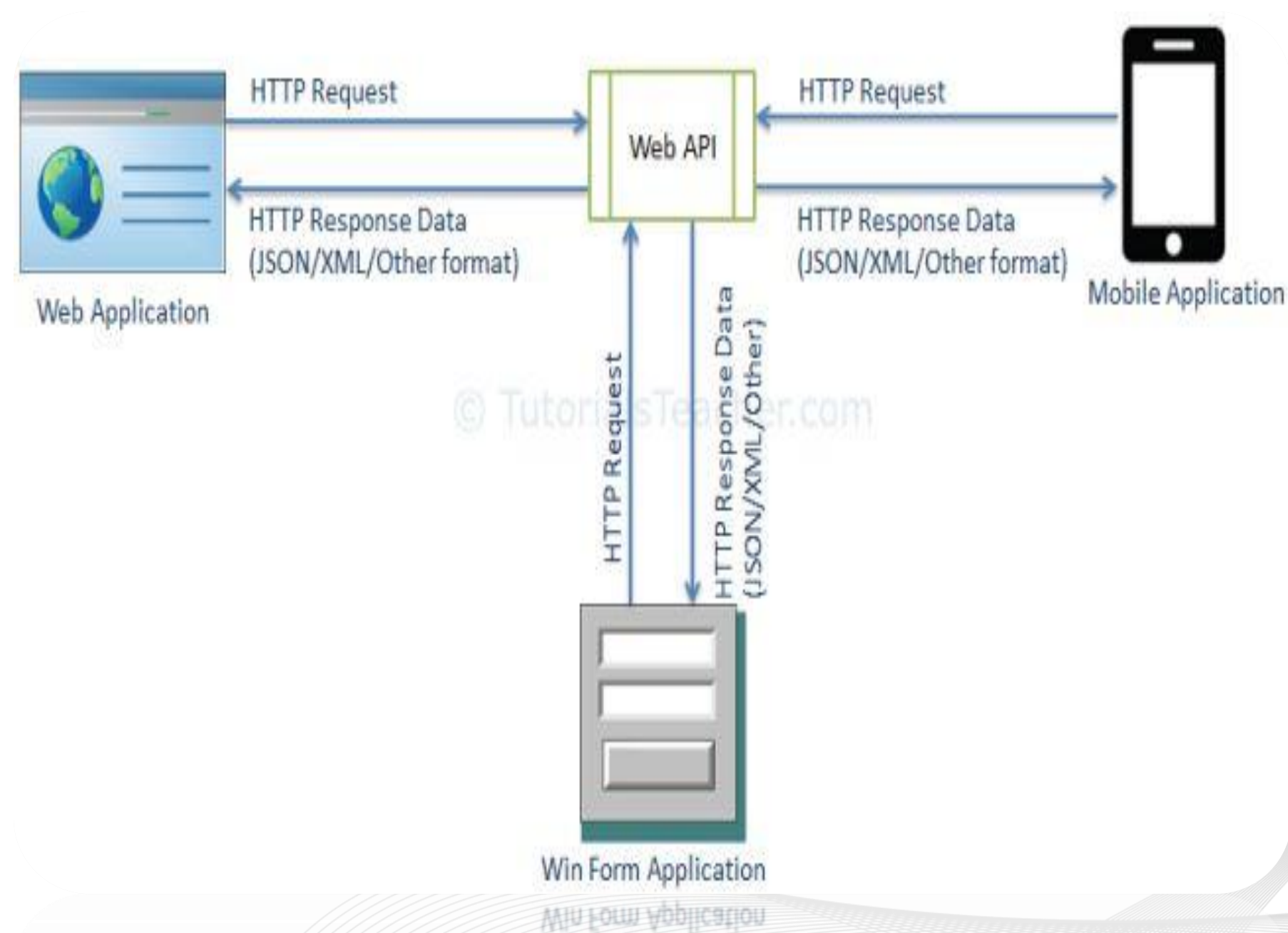
□ Framework que facilita a construção de serviços **over HTTP** com forte interoperabilidade e possibilidade de acesso a partir dos mais variados dispositivos

- Permite que sistemas de diferentes fabricantes, ou mesmo sistemas diferentes dentro da mesma organização, possam interagir, trocando dados e informações de forma eficiente.



- ❑ Arquitectura simplificada
- ❑ Utiliza rotas para disponibilizar recursos
- ❑ Lembra o desenvolvimento com Controllers(MVC)
- ❑ Leve
- ❑ Testável
- ❑ Escalável
- ❑ Open Souce







A photograph of a modern campus entrance. In the foreground, a large yellow rectangular sign with the word "ISPTEC" in white capital letters sits on a grassy area. Behind it, a tall silver pole supports a large, abstract yellow sculpture that resembles a stylized infinity symbol or a continuous loop. To the right, a modern white building with a flat roof and large glass windows is visible. A paved walkway leads towards the building. In the background, there are trees, a fence, and a clear blue sky. A semi-transparent grey banner with white text is overlaid across the middle of the image.

ATÉ A PRÓXIMA AULA!

Volenti Nihil Difficili - “A quem quer, nada é difícil”